

Foundations of Artificial Intelligence - 2026.1

Sequential Decision Making Under Certainty

Felix Mohr

Universidad de La Sabana

Chía, February 16, 2026

Course Outline

1. Graph Search Problems
2. Graph Search when Solution Quality is Path-Independent
3. Graph Search when Solution Quality is Path-Dependent (Uniform Cost Search)

Graph Search Problems

We now assume that the environment is sequential.

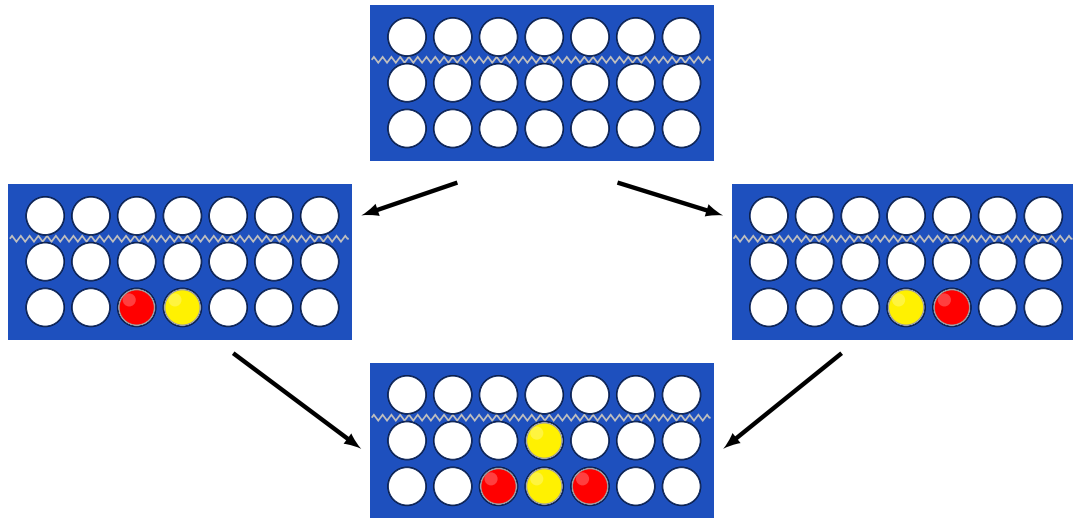
We can then think of the environment as a graph with S being the nodes and an edge (s, s') if there is an event e so that $s' = \psi(s, e)$. Some states are desirable goal states.

In most sequential environments, a state can be reached in different ways:

- The current location in a tour planning problem.
- The state of a game.

Graph Search Problems

Example: Connect 4 with fixed yellow second player



Graph Search Problems

The Space of Paths as the Search Space

Formally, we denote the **search graph** as $G = \langle N, A \rangle$ described via:

- a distinguished initial node $n_0 \in N$
- a **successor function** $\text{succ}_G : N \rightarrow 2^N$ s.t. $(n, n') \in A$ iff $n' \in \text{succ}_G(n)$

The fact that succ_G is a function allows to describe even infinite graphs compactly.

A node marking **goal predicate** $\star_G : N \rightarrow \{0, 1\}$ marks desirable nodes with 1.

Let $\mathcal{P}_G$ be the set of all paths in G . Then $\mathcal{X} = \{\mathbf{p} \in \mathcal{P}_G \mid (\mathbf{p}_0 = n_0) \wedge (\star_G(\mathbf{p}_{-1}) = 1)\}$

Graph Search Problems

Compiling a Problem Instance for Simple Constructive Search

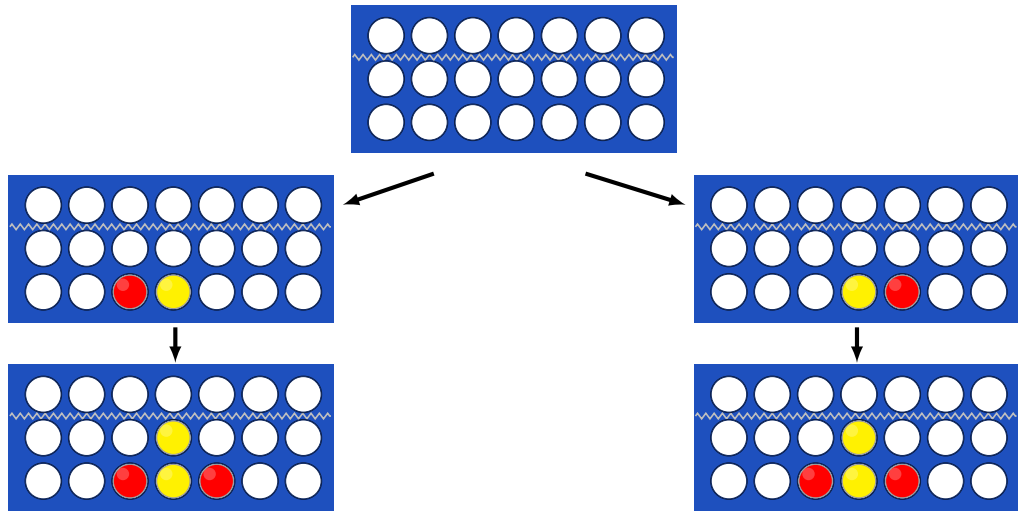
Since a path is a sequence of edges (n, n') , we can think of edges as operations to construct paths in a constructive search, i.e., $O = A$ and hence $\mathcal{P}_G \subset O^*$.

Then we could apply Simple Constructive Search using

- $\mathcal{W} := \{\mathbf{p} \in \mathcal{P}_G \mid \mathbf{p}_0 = n_0\}$
- $\text{succ} := \{\mathbf{p} \oplus n \mid n \in \text{succ}_G(\mathbf{p}_{-1})\}$
- $\star(\mathbf{p}) := \star_G(\mathbf{p}_{-1})$

Graph Search Problems

Example: Connect 4 with fixed yellow second player



Node-Based Graph Search

Efficiency Problems of Simple and General Constructive Search - Double Expansion

In General Constructive Search, every path starting in n_0 will be added to *OPEN*.

This causes two problems if the underlying graph is not a tree:

- 1 If there are k ways to reach a node n , then k paths to n will end up on *OPEN* over time, all of which with the same **rest problem** to find a goal node from n .

Example: Different move sequences in Connect-4 against a static player.

- 2 If the graph has cycles, then the algorithm will never terminate.

Example: A tour planning problem when we want to get from A to B .

Graph Search Problems

If we have two paths to the same node, it seems we could just keep one of them. While sometimes we can drop any of the two, this is not always optimal:

- In Connect-4 with deterministic opponent, it doesn't matter *how* we reach a state, and we can just drop one of the paths.
- Finding a shortest tour from A to B , in two tours that both go over C , it clearly matters *how* we reached C . We should drop the path with the costlier route to C .

Only because $\mathbf{p}$ and $\mathbf{p}'$ have the same rest problem, we cannot arbitrarily drop any of them, because completions from $\mathbf{p}$ may have other quality than those of $\mathbf{p}'$.

Graph Search Problems

If we want to drop paths, we need two conditions:

- 1 the pre-order $\succeq$ of $\mathcal{X}$ extends to partial paths in $\mathcal{W} \setminus \mathcal{X}$
- 2 for two partial paths $\mathbf{p}, \mathbf{p}'$ with same rest problem and $\mathbf{p} \succeq \mathbf{p}'$, for any goal path $\mathbf{p}'^*$ obtained from $\mathbf{p}'$, there is a goal path $\mathbf{p}^*$ complete from $\mathbf{p}$ so that $\mathbf{p}^* \succeq \mathbf{p}'^*$

Then if $\mathbf{p} \succeq \mathbf{p}'$ and both have the same rest problem, there is no reason to keep $\mathbf{p}'$.

Also, under these conditions it seems that we could use $\succeq$ to sort paths on *OPEN*.

Since General Constructive Search does not compare/drop items from *OPEN* except when expanding, we need a modification of the algorithm.

Course Outline

1. Graph Search Problems
2. Graph Search when Solution Quality is Path-Independent
3. Graph Search when Solution Quality is Path-Dependent (Uniform Cost Search)

Graph Search when Solution Quality is Path-Independent

In this section, we assume that the quality of any solution path $\mathbf{p}^*$ can be computed only from the last node $\mathbf{p}_{-1}^*$ without knowing the rest of the path.

This condition holds in various scenarios:

- When we play a game if we only care about winning (but not winning *fast*).
 - ▶ Connect-4 against a fixed and known player
 - ▶ Solving a standard puzzle
- In construction/configuration tasks:
 - ▶ Building a House
 - ▶ Molecular Structure Optimization
 - ▶ Machine Learning Pipelines

Graph Search when Solution Quality is Path-Independent

Graph Search - Motivation

In this setting, we can ignore any new path $\mathbf{p}$ to a node n after the first one we found.

Formally, we should not put a path $\mathbf{p}$ on *OPEN* if there is a $\mathbf{p}'$ with $\mathbf{p}_{-1} = \mathbf{p}'_{-1}$ and

- 1 $\mathbf{p}' \in \text{OPEN}$. We can simply check this.
- 2 $\mathbf{p}'$ was on *OPEN* in the past.

Since a path is only removed from *OPEN* if it is expanded, we can use a list *CLOSED* with all nodes that were the head of a path ever taken from *OPEN*.

Then check whether $\mathbf{p}_{-1} \in \text{CLOSED}$.

Graph Search when Solution Quality is Path-Independent

Equality Test for Nodes

The equality and membership tests $=$ and $\in$ introduces a new dynamic to the situation:

We now need to be able to test node objects on **equality**.

In practice, the class we use to implement the nodes should have methods to

- **hash** objects (to put nodes into bins), e.g., to organize a set, and
- check two node objects for equality

Graph Search when Solution Quality is Path-Independent

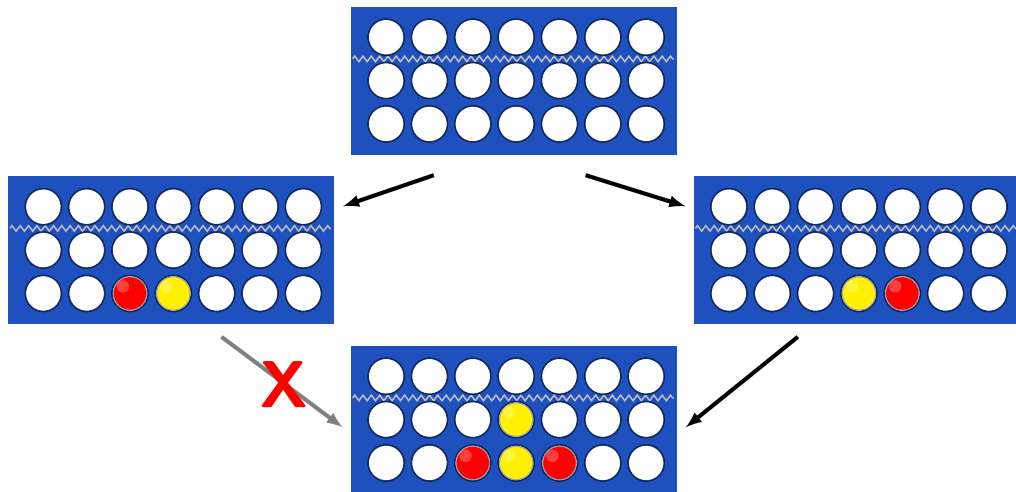
Require: $(\langle n_0, succ_G, \star_G, \succ \rangle)$

Ensure: solution or failure

```
1:  $OPEN \leftarrow \{(n_0)\}$ ,  $p^* = \perp$ ,  $CLOSED \leftarrow \emptyset$ 
2: while  $OPEN \neq \emptyset$  do
3:    $p \leftarrow \text{Remove-First}(OPEN)$ 
4:    $CLOSED \leftarrow CLOSED \cup \{p\}$ 
5:   for  $n' \in succ_G(p)$  if  $n' \notin CLOSED \wedge \nexists \tilde{p} \in OPEN : \tilde{p}_{-1} = n'$  do
6:      $p' \leftarrow p \oplus (n')$ 
7:     if  $\star_G(n')$  then
8:       if  $\succ$  is None then return  $p'$ 
9:       else if  $(p^* = \perp) \vee (p' \succ p^*)$  then  $p^* \leftarrow p'$ 
10:    else
11:       $OPEN \leftarrow OPEN \cup \{p'\}$ 
12:    end if
13:  end for
14: end while
15: return  $p^*$ 
```

Graph Search when Solution Quality is Path-Independent

Example: Connect 4 with fixed yellow second player



Graph Search when Solution Quality is Path-Independent

Computational Inefficiencies of Plain Lists

If we use plain lists as workspace items, we have two computational inefficiencies:

- 1 We are wasting a lot of memory: If a reference to a node objects costs b bytes, then storing a path $\mathbf{p}$ consumes $b|\mathbf{p}|$ bytes.

But many paths share the same nodes, which could be exploited.

- 2 Since we do not store the events that cause the transition from one node to another, plan recovery can be cumbersome.

For example, for Connect-4, we have to compare two subsequent boards to see in which cell the chip was placed.

Graph Search when Solution Quality is Path-Independent

We can elegantly address both problems by introducing an internal data structure `Path`.

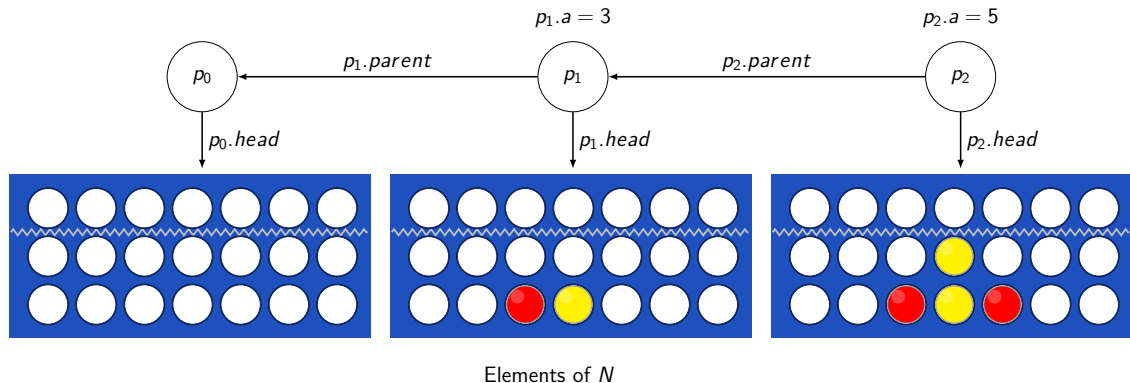
Think of `Path` as a class with three fields:

- a pointer $p.head$ to the node associated with the head n
- a pointer $p.parent$ to the path whose head is the parent of n
- a field $p.a$ describing the *action* taken in $n.parent$ to reach n

A `Path` has two constructors:

- 1 one only with a node (used with n_0) and
- 2 one with an existing parent, a new head node, and the action taken in the parent.

Graph Search when Solution Quality is Path-Independent



Graph Search when Solution Quality is Path-Independent

Changes in the Code

To properly use this structure in code, we need to adjust the code in three places:

- 1 The succ_G function previously returned neighboring nodes, but we now also need the action (event) that led us there.

We adjust succ_G to comply with the condition $(a, n') \in \text{succ}_G(n)$ if $\psi(n, a) = n'$.

- 2 Since the path is now an object and not a list anymore, we need to refer to $p.\text{head}$ wherever we previously used $\mathbf{p}_{-1}$, which semantically is the same.
- 3 We do not return the `Path` object, which is an item in $\mathcal{W}$, but a list of actions.

Graph Search when Solution Quality is Path-Independent

General Graph Search - Algorithm

Require: $\langle \langle n_0, succ_G, \star_G, \succ \rangle \rangle$

Ensure: solution or failure

```
1:  $OPEN \leftarrow \{\text{new Path}(n_0)\}$ ,  $p^* = \perp$ ,  $CLOSED \leftarrow \emptyset$ 
2: while  $OPEN \neq \emptyset$  do
3:    $p \leftarrow \text{Remove-First}(OPEN)$ 
4:    $CLOSED \leftarrow CLOSED \cup \{p.head\}$ 
5:   for  $(a, n') \in succ_G(p.head)$  if  $n' \notin CLOSED \wedge \nexists \tilde{p} \in OPEN : \tilde{p}.head = n'$  do
6:      $p' \leftarrow \text{new Path}(p, n', a)$ 
7:     if  $\star_G(n')$  then
8:       if  $\succ$  is None then return  $\langle p'.a \text{ for } p' \in p \rangle$ 
9:       else if  $(p^* = \perp) \vee (p' \succ p^*)$  then  $p^* \leftarrow p'$ 
10:    else
11:       $OPEN \leftarrow OPEN \cup \{p'\}$ 
12:    end if
13:  end for
14: end while
15: return  $\langle p'.a \text{ for } p' \in p^* \rangle$ 
```

Course Outline

1. Graph Search Problems
2. Graph Search when Solution Quality is Path-Independent
3. Graph Search when Solution Quality is Path-Dependent (Uniform Cost Search)

Graph Search when Solution Quality is Path-Dependent

Decomposing Cost Measures

We now assume that the ordering $\succeq$ on paths in $\mathcal{X}$ is induced by a **cost function**

$$c : \mathcal{X} \rightarrow \mathbb{R}_+$$

so that $x \succeq x' \stackrel{\text{def}}{\iff} c(x) \leq c(x')$.

In the above form,

- $c(x)$ can depend on the whole path x , not only the last node.

This means we cannot longer discard paths to nodes we have already discovered.

- c is only defined on solution paths, and we have no cost for other paths.

This means we cannot compare cost of two paths to some intermediate node n .

Graph Search when Solution Quality is Path-Dependent

Decomposing Cost Measures

How can we extend $c : \mathcal{X} \rightarrow \mathbb{R}_+$ to $c : \mathcal{P} \rightarrow \mathbb{R}_+$ over all paths in G ?

c **decomposes** along edges iff for any paths $\mathbf{p1}, \mathbf{p2}$ pointing at n and any $\mathbf{p}'$ we have that

$$c(\mathbf{p1} \oplus \mathbf{p}') - c(\mathbf{p1}) = c(\mathbf{p2} \oplus \mathbf{p}') - c(\mathbf{p2})$$

A path cost measure c decomposes along edges iff it is additive, that is, for any $\mathbf{p}$:

$$c(\mathbf{p}) = \sum_{i=1}^{|\mathbf{p}|-1} c((\mathbf{p}_i, \mathbf{p}_{i+1}))$$

Graph Search when Solution Quality is Path-Dependent

When is it reasonable to assume decomposing path costs?

Path costs do *not* decompose iff adding a path $\mathbf{p}'$ to $\mathbf{p1}$ incurs a different cost increase than adding it to $\mathbf{p2}$ even though the rest problem of $\mathbf{p1}$ and $\mathbf{p2}$ is the same.

A non-decomposing cost measure would require that solving the *same* rest problem n (path heads) in a specific way adds different costs depending on how we *arrived* at n .

Example: Arriving “late” at n imposes traffic jam and higher cost for the same rest tour.

If we can exclude this, we can safely assume that c is additive.

Graph Search when Solution Quality is Path-Dependent

Memorizing path cost with Path Objects

From here on,

- we assume that path costs decompose and hence are additive.
- paths p, p' , etc. are `Path` objects

Then we can associate each `Path` object p with the total costs from n_0 to $p.head$:

$$p.g := p'.g + c(p'.head, p.head)$$

where $p' = p.parent$.

Due to the recursive definition, $p.g$ stores costs of the *path*.

Graph Search when Solution Quality is Path-Dependent

Parent Discarding

Suppose that we check to expand a path p to a node n' , but there is already a path $\tilde{p} \in OPEN$ with $\tilde{p}.head = n'$.

Given that we want to find a *cheapest* path, we may want to consider to *update* the path to n' by *changing* the parent pointer of $\tilde{p}$ from $\tilde{p}.parent$ to p if

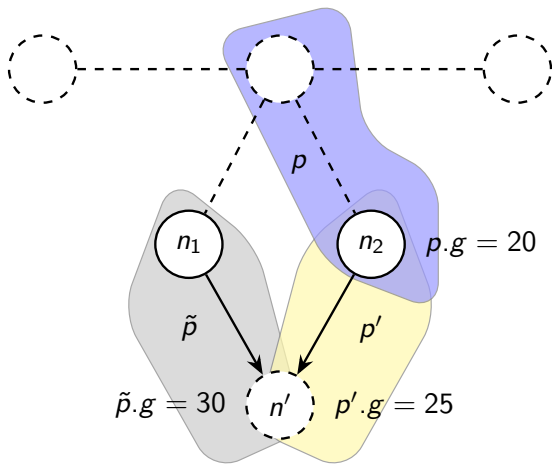
$$\tilde{p}.g > p.g + c(p.head, n')$$

and also update $\tilde{p}.g$ respectively. This is called **parent discarding**.

Otherwise, we would discard the *new* path via $p.head$.

Graph Search when Solution Quality is Path-Dependent

Parent Discarding - Example



Graph Search when Solution Quality is Path-Dependent

Using Path Cost to Steer the Search

We can use g not only to maintain the best path to each node.

Since g is defined for *all* paths (also those not ending in a goal node), we can use g to *order* the paths in *OPEN*.

This allows us to search the graph in a more informed way than before.

Graph Search when Solution Quality is Path-Dependent

Uniform-Cost Search - The Algorithm

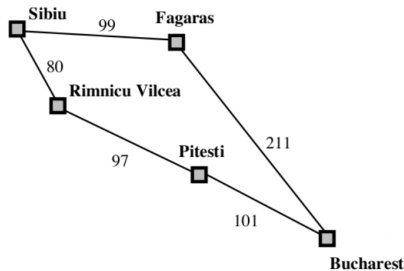
Require: $(\langle n_0, succ, *, c \rangle)$

Ensure: solution or failure

```
1:  $OPEN \leftarrow \{\text{new Path}(n_0)\}, \quad CLOSED \leftarrow \emptyset$ 
2: while  $OPEN \neq \emptyset$  do
3:    $p \leftarrow \text{Remove-First}(OPEN)$ 
4:   if  $*(p.head)$  then return  $\langle p'.a \text{ for } p' \in p \rangle$ 
5:    $CLOSED \leftarrow CLOSED \cup \{p.head\}$ 
6:   for  $(a, n') \in succ(p.head)$  if  $n' \notin CLOSED$  do
7:      $g \leftarrow p.g + c(p.head, n')$ 
8:     if  $\nexists \tilde{p} \in OPEN : \tilde{p}.head = n'$  then
9:        $OPEN \leftarrow OPEN \cup \{\text{new Path}(p, n', a, g)\}$ 
10:    else if  $\tilde{p}.g > g$  then
11:       $\tilde{p}.parent \leftarrow p$ 
12:       $\tilde{p}.g \leftarrow g$ 
13:       $\tilde{p}.a \leftarrow a$ 
14:    end if
15:  end for
16: end while
```

Graph Search when Solution Quality is Path-Dependent

Uniform-Cost Search - An Example: Go from Sibiu to Bucharest



- 1 Expand Sibiu, generating Rimnicu $g = 80$, Fagaras $g = 99$.
- 2 Expand Rimnicu, generating Pitesti $g = 80 + 97 = 177$ (and Sibiu, which is pruned).
- 3 Expand Fagaras, generating Bucharest $g = 99 + 211 = 310$
- 4 Expand Pitesti, generating Bucharest $g = 177 + 101 = 278$; Replace Bucharest $g = 310$ with Bucharest $g = 278$ in *OPEN*!
- 5 Expand Bucharest $g = 278$.

Graph Search when Solution Quality is Path-Dependent

The Traversal Tree

The $p.parent$ links in the path induce a tree structure, which we call **traversal tree**.

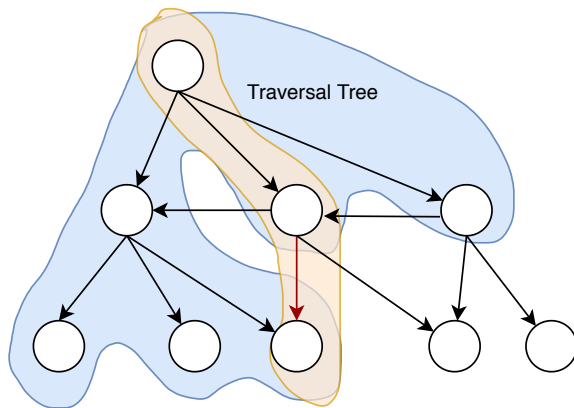
At each point of time, the algorithm has present *at most one* path between two vertices of the search graph even if the underlying graph has more.

Even if we have two nodes n and n' in *CLOSED* that are connected in the original graph, it doesn't mean that the traversal tree contains the edge (n, n') .

In General Constructive Search, the traversal tree grows monotonically, but in Uniform Cost Search, edges may disappear from the traversal tree (due to parent discarding).

Graph Search when Solution Quality is Path-Dependent

The Traversal Tree



All nodes of the orange path are contained in the traversal tree, but the second edge is not part of it. It is also useful to mark closed nodes for clarity (no ambiguities here).

Graph Search when Solution Quality is Path-Dependent

Uniform-Cost Search - Formal Properties when Applied to Finite Graphs

We applied parent discarding for paths on *OPEN* if they were more expensive than a newly discovered path to n' .

What if no path to n' is on *OPEN* but we already expanded such a path earlier?

Then n' is in *CLOSED*, and other paths *containing* n' are on *OPEN*.

One can prove for Uniform Cost Search that it is not possible that the new path to n' is cheaper than the one that was already expanded, and we can discard it.

Graph Search when Solution Quality is Path-Dependent

Uniform-Cost Search - Formal Properties when Applied to Finite Graphs

Suppose that:

- 1 every node has only a finite number of successors (locally finite);
- 2 $\exists \varepsilon > 0 : c(n, n') \geq \varepsilon$ for all n and $n' \in \text{succ}(n)$; and
- 3 there is a solution path.

Then Uniform-Cost Search will (i) terminate and (ii) return an optimal solution path.

Summary

Summarizing the class in one sentence, we have learned that

- solving sequential decision making problems is afflicted with inefficiencies when being solve with General Constructive Search;
- we can reduce these inefficiencies by *allowing only one path to each node* as long as path quality only depends on the final node; and
- we can reduce the inefficiencies even further through *parent discarding* as long as path costs are additive.